Devoid Viewer

Codebase Documentation & Design Guide

This document explains every file, struct, method, and design decision in the Devoid Viewer codebase. It is written for someone new to Rust who wants to understand not only *what* the code does but *why* it was written that way — including the threading model, the choice of external libraries (crates), and the subtleties of building a responsive GUI application.

Table of Contents

Section	Topic	Page
1	Project structure overview	2
2	Cargo.toml — dependencies explained	2
3	nav.rs — directory navigation	5
4	loader.rs — image decoding	7
5	main.rs — application core	9
6	Threading & asynchronous design	17
7	Key design decisions & trade-offs	19
8	Caveats & known limitations	21
9	Glossary of Rust concepts used	22

1. Project Structure Overview

The project follows the standard Cargo layout:

File	Responsibility
`Cargo.toml`	Project name, version, and all external dependencies.
`build.rs`	Build script — embeds a Windows icon and metadata via winres.
`src/main.rs`	Event loop, app state, input handling, pixel renderer, window management.
`src/loader.rs`	Image decoding for all supported formats. Owns Frame and DecodedImage.
`src/nav.rs`	Directory scanning, human-sorted sibling list, next/prev navigation.

2. Cargo.toml — Dependencies Explained

Each crate was chosen deliberately for leanness. The dependency graph is roughly 50–70 crates and the release binary around 2–3 MB, compared to 200+ crates and ~9 MB with a full GUI toolkit.

Windowing: winit

winit is the de-facto standard Rust crate for OS windows and input events. It handles the event loop, keyboard, mouse, window resize, drag-and-drop, and monitor queries. It has no rendering opinion — it gives you a window handle and events. The rwh_05 feature enables the raw-window-handle v0.5 interface that softbuffer requires.

Pixel display: softbuffer

softbuffer provides a Vec<u32> mapping directly to the window's pixels. You write colours and call present(). No GPU pipeline, no shader compilation, no texture atlas. On Windows it uses a GDI DIB section, compositor-accelerated without a GPU context.

> ■ *Known limitation: the DWM compositor fills every new window with white before the application paints. This causes a brief white flash on startup. This is a fundamental Win32 property — the default WNDCLASS background brush is white and DWM applies it before any softbuffer present reaches the screen. Multiple approaches to eliminate it were attempted (with_transparent, with_no_redirection_bitmap, NULL_BRUSH via SetClassLongPtrW) — each caused other problems. The flash is minimised by decoding the first image synchronously and calling request_redraw() before the event loop starts.*

Native dialogs: windows-sys

windows-sys is a zero-overhead Windows API binding. We enable Win32_UI_WindowsAndMessaging for MessageBoxW and SystemParametersInfoW(SPI_GETWORKAREA), and Win32_Foundation for the RECT type. It is a target-specific dependency and does not affect non-Windows builds.

Image decoding: image

The **image** crate supports BMP, PNG, JPEG, GIF, WebP, TIFF, and ICO. Features enabled selectively. GIF animation uses image::codecs::gif::GifDecoder and IntoFrames.

JPEG XL: jxl-oxide

JPEG XL is not yet in the image crate. **jxl-oxide** is a pure-Rust JXL decoder. We use the low-level streaming API because the higher-level image feature had a private-method issue in version 0.10.

SVG: resvg + usvg + tiny-skia

- **usvg** parses and normalises the SVG scene tree.
- **resvg** renders the tree into a tiny-skia Pixmap.
- **tiny-skia** rasterises to premultiplied RGBA, which we unpremultiply.

Channel communication: crossbeam-channel

Faster and more ergonomic than std::sync::mpsc. Bounded at capacity 32 to throttle background threads if the UI falls behind.

Windows resource embedding: winres (build dependency)

Runs at build time only. Embeds a .ico and version metadata into the .exe. Zero runtime cost.

3. nav.rs — Directory Navigation

Maintains a human-sorted list of sibling image files in the same directory as the currently open file. Navigation wraps at both ends.

struct DirNavigator

- **files: Vec<PathBuf>** — sorted sibling list.
- **index: usize** — position of the current file.

Derives **Default** — auto-generated empty initialiser.

fn set_current(path)

Called on every image open. Rebuilds the sibling list only when the directory changes. Falls back to a single-entry list if the path is not found in the scan (e.g. different drive).

fn rebuild(dir)

Reads the directory, filters for supported files, sorts using **human_sort** so img2 < img10 rather than img10 < img2.

fn next() / fn prev()

Return None if fewer than 2 images exist — a single-image directory produces no navigation, preventing a redundant reload of the same file. When multiple images exist, both methods wrap around and return the new path. Use checked_sub(1) in prev() to safely handle the unsigned integer wrap.

fn peek_next() / fn peek_prev()

Read-only lookahead used by preload_adjacent() to decode neighbours before the user navigates to them. Also returns None for single-image directories.

fn remove(path)

Removes a path after deletion. Uses saturating_sub to keep the index in bounds on an empty list.

fn human_sort / fn consume_number

Character-by-character comparator. Digit runs are compared numerically by consume_number(), ensuring img2 < img10.

4. loader.rs — Image Decoding

Owns the image type definitions and all decoding logic. Returns **DecodedImage** directly — no conversion needed in main.rs.

struct Frame

pixels: Vec<u8> — straight RGBA8, row-major. **delay_ms: u32** — frame duration (0 for non-animated formats).

struct DecodedImage

One or more frames sharing the same dimensions. **is_animated()** returns true when frames.len() > 1. **frame(idx)** returns a reference to a specific frame. **single()** is a private convenience constructor for single-frame images.

fn load_image(path) → LoadResult

Dispatches by lowercased extension: svg → load_svg, jxl → load_jxl, gif → load_gif, everything else → load_standard.

fn load_standard — BMP, PNG, JPEG, WebP, TIFF, ICO

image::open() decodes and normalises to RGBA8. **pixels_from_dynamic** extracts raw bytes and dimensions.

fn load_gif — Animated GIF

Uses GifDecoder which implements AnimationDecoder. Dimensions are read from the decoder before it is consumed into frames — a single file open. Each frame's delay comes from frame.delay().numer_denom_ms() with a 20ms minimum and 100ms fallback. Frames with zero delay are treated as 100ms.

fn load_jxl — JPEG XL

- BufReader for efficient I/O.
- request_color_encoding(srgb(...)) forces sRGB output.
- render_frame(0) decodes frame 0 only.
- Stream yields interleaved f32. Channel count 1/2/3/4 expanded to RGBA u8 via **f32_to_u8** which clamps to [0,1] then scales to [0,255].

fn load_svg — SVG

- **usvg** parses and normalises the scene tree.
- **resvg** renders to a tiny-skia Pixmap (premultiplied RGBA).
- Unpremultiply: divide each channel by alpha. Zero-alpha pixels zeroed.

5. main.rs — Application Core

Contains the winit event loop, all application state, and the pixel renderer. No widget toolkit, no font renderer, no GPU pipeline.

5.1 const BG: u8 = 42

The background grey used for the blank state and alpha compositing. Named once, used in render() and composite().

5.2 enum Msg

Four variants from background threads to the UI thread:

- **Loaded** — primary decode complete: path, image, resize flag.
- **Preloaded** — speculative decode: path and image for the cache.
- **Error** — decode failed: path and error string.
- **Deleted** — async deletion result: path and Ok(())/Err(String).

All contained types implement Send automatically — no unsafe annotation needed.

5.3 struct WakeUp / type Proxy

WakeUp is a zero-size marker struct used as the custom user-event type for EventLoop<WakeUp>. Background threads call proxy.send_event(WakeUp) to wake the sleeping event loop. **Proxy** is a type alias keeping the verbose generic manageable.

5.4 struct View<'a>

A snapshot of what the renderer needs: image reference, current frame index, and pan offset. App::view() produces one. Passing View to render() keeps the function signature to four arguments rather than many, and prevents the renderer from mutating app state. The 'a lifetime ensures the View cannot outlive the App.

Note: **zoom has been removed from View**. Zoom was only ever changed by the scroll-wheel handler, which has been removed. The fit-to-window scale is now computed directly in render() from image and window dimensions — no zoom multiplier.

5.5 enum OpenMode

FirstImage — resize the window to the image. **Navigate** — keep the existing window size. An enum instead of a bool makes every call site self-documenting.

5.6 struct App — fields

- **current_image** — not cleared when navigation starts. The previous image stays visible until the new one is decoded.
- **frame_index / next_frame_at** — GIF animation state.

- **pan / drag_start / pan_origin** — pan state. Reset on each new image. Note: zoom field has been removed entirely.
- **loading_path** — path being decoded. Used to discard stale channel messages if the user navigated away.
- **preload_cache** — decoded neighbours in CPU memory, ready for instant display. No GPU upload step.
- **deleting** — prevents double-deletion while async remove_file is in flight.
- **pending_center** — triggers centering on the event after a resize has been applied (two-event sequence, see Section 6.7).

5.7 fn view() → Option<View>

Produces a View snapshot from current state, or None if no image is loaded. Called once per frame and passed to render().

5.8 fn open_image()

Checks the preload cache for an instant hit. Otherwise sets loading_path, updates the navigator, and spawns a background thread that calls load_image(), constructs a Msg, sends it, and fires WakeUp.

5.9 fn apply_image() → Option<(u32, u32)>

Stores the image, resets pan, resets animation, updates the navigator, starts preloading neighbours. Returns Some((w,h)) when the window should resize — a return value rather than a side-effect field.

5.10 fn clear_image()

Resets current_image, current_path, pan, and animation state. Calls **reset_pan()** and **reset_animation()** rather than duplicating their field assignments.

5.11 fn preload_adjacent()

Spawns up to two threads for next and previous siblings. Called from apply_image() — they are a logical unit.

5.12 fn reset_animation() / fn tick_animation()

reset_animation() sets frame_index to 0 and computes next_frame_at. Called from both apply_image() and clear_image() — one place for the logic.

tick_animation() checks if the current deadline has passed, advances frame_index, schedules the next deadline, returns true if a redraw is needed. Uses if-let to safely handle the case where current_image is None.

5.13 fn reset_pan() / fn start_drag() / fn update_drag() / fn end_drag()

reset_pan() zeroes pan and clears drag_start. Called on every new image and on Backspace/Home/middle-click.

start_drag() records the cursor position and current pan as the drag origin. **update_drag()** computes the new pan from the delta and returns true if dragging is active (so the caller can request a redraw without a separate is_some() check). **end_drag()** clears drag_start.

5.14 fn navigate(forward: bool)

Calls navigator.next() or navigator.prev() — both return None when fewer than 2 images exist, so navigation on a single-image directory is a no-op.

5.15 fn request_delete() / fn on_deleted()

request_delete() sets deleting=true and spawns a background thread that calls remove_file() and sends Msg::Deleted.

on_deleted() clears the flag, removes the path from the navigator, clears the preload cache, and navigates to the next sibling or clears the view.

5.16 fn drain_messages() → Option<(u32, u32)>

Non-blocking try_recv() loop. Four clean match arms, each doing one thing. Propagates resize dimensions as a return value to the event loop.

5.17 Rendering functions

- **fit_scale()** — min(win_h/img_h, win_w/img_w). The image fills the window in its dominant dimension without cropping.
- **nearest()** — point sample: read one source pixel at the integer-truncated source coordinate. Fast and correct for fit-to-window scale where each destination pixel maps to approximately one source pixel.
- **composite()** — alpha-blends a straight-RGBA pixel over BG. Inner blend() function shares the formula across R, G, B.
- **render()** — takes Option<View>. Fills with BG if None. Otherwise computes scale and destination rect, iterates every window pixel, samples with nearest(), composites, writes to the buffer.

Note: **bilinear() and lerp_u8() have been removed**. They existed to support zoom-in rendering, but since scroll-wheel zoom has been removed, zoom is always 1.0 and bilinear provides no visible benefit over nearest-neighbour at fit-to-window scale.

5.18 fn confirm_delete_dialog() / fn show_error_dialog()

Both call MessageBoxW from windows-sys. Blocking — correct for modal confirmation. UTF-16 via encode_wide().chain(Some(0)). #[cfg(not(windows))] fallbacks for cross-platform compilation.

5.19 fn work_area() / fn windows_work_area() / fn clamp_to_work_area() / fn center_window()

work_area() returns usable desktop dimensions. On Windows delegates to **windows_work_area()** which calls SPI_GETWORKAREA for the exact taskbar-excluded rect. On other platforms uses 90%×85% of the monitor size. Separated into its own #[cfg(windows)] function returning Option so

work_area() falls through cleanly to the cross-platform path.

clamp_to_work_area() scales image dimensions down to fit within the usable area. **center_window()** positions the window at the monitor centre with a minimum Y offset of 30×scale_factor pixels so the title bar is always reachable.

5.20 fn main()

- **Synchronous first decode:** load_image() runs on the main thread before the event loop. The window opens at the correct size and apply_image() is called directly.
- **window.request_redraw() before event_loop.run():** queues a redraw immediately so the first frame is painted on the very first event loop iteration.
- **mark_dirty closure:** sets dirty=true and calls request_redraw(). Replaces a repeated two-line pattern in every event handler.
- **GIF control flow:** ControlFlow::WaitUntil(next_frame_at) when animating; ControlFlow::Wait otherwise. Zero CPU when idle.

6. Threading & Asynchronous Design

6.1 Why threads?

Image decoding is I/O-bound (disk reads) and CPU-bound (decompression, SVG rasterisation). A large image can take hundreds of milliseconds. Doing this on the event loop thread would freeze the window entirely.

6.2 The channel pipeline

- One bounded crossbeam channel, capacity 32, created in main().
- The Sender (msg_tx) is cloned for each background thread — cheap.
- The Receiver (msg_rx) is drained in drain_messages() on each Event::UserEvent(WakeUp).
- Background threads post a Msg then call proxy.send_event(WakeUp) to inject a synthetic event and wake the sleeping loop immediately.

■ *proxy.send_event() is thread-safe — EventLoopProxy is Arc-based.*

6.3 ControlFlow::Wait — zero idle CPU

The event loop sleeps until an event arrives. When a GIF is playing, ControlFlow::WaitUntil(next_frame_at) wakes precisely at the next frame deadline. When idle, the process uses effectively zero CPU.

6.4 Synchronous first image

The first image is decoded on the main thread before the event loop starts. The window is created at the correct size and apply_image() is called directly — no WakeUp round-trip. Subsequent navigation remains async.

6.5 Stale result discarding

If the user navigates faster than images decode, multiple Loaded messages may arrive. Each is checked against loading_path: mismatches are discarded, preventing the wrong image from appearing.

6.6 Async deletion

remove_file() runs on a background thread. The deleting flag prevents double-deletion. The result arrives as Msg::Deleted and is handled in drain_messages().

6.7 The two-event resize/centre sequence

window.request_inner_size() is processed asynchronously by the OS. Calling center_window() in the same event handler reads the pre-resize size and computes the wrong position. Fix: on the WakeUp that delivers the first image, issue request_inner_size and set pending_center=true. On the next WakeUp the resize has landed and center_window() is accurate.

7. Key Design Decisions & Trade-offs

winit + softbuffer over egui/eframe

egui/eframe is a full immediate-mode GUI toolkit with font atlas, widget layout, and OpenGL backend. For a viewer displaying one image and responding to a handful of keys this was over-engineering: ~200+ crates and ~9 MB. winit + softbuffer: ~50-70 crates and ~2-3 MB. Trade-off: no widgets, native OS dialogs required.

Frame and DecodedImage in loader.rs

Earlier versions had RawImage in loader.rs and DecodedImage in main.rs with a From conversion between identical structs. Moving both into loader.rs eliminated the conversion: load_image() returns DecodedImage directly.

zoom removed, nearest-neighbour only

Scroll-wheel zoom was implemented but not wanted. Removing it eliminated the zoom field, zoom_at() method, ZOOM_MIN/MAX constants, the bilinear sampling path, lerp_u8(), and the use_bilinear branch in render(). Nearest-neighbour is correct and fast for fit-to-window scale where each destination pixel maps to approximately one source pixel.

Single-image navigation is a no-op

nav.rs next() and prev() return None when files.len() < 2. Previously they returned Some with the same path, causing a redundant full reload of the current image. The fix is in the navigator, not the event handler.

clear_image() calls reset_pan() and reset_animation()

Earlier, clear_image() manually zeroed frame_index and next_frame_at separately from reset_animation(). Now it delegates to the named helpers, keeping the reset logic in one place each.

Return values over side effects

apply_image() returns Option<(u32,u32)> and drain_messages() propagates it. The event loop receives resize intent as a value — the App struct has no window-management fields except pending_center.

Native OS dialogs

MessageBoxW is blocking — exactly correct for modal confirmations. Requires zero rendering code, inherits the system theme, and eliminates the need to suppress navigation input during the dialog.

8. Caveats & Known Limitations

Startup white flash (known, unfixed)

On Windows, DWM fills every new window's client area with white before the application paints. This is a fundamental Win32 property — the default WNDCLASS background brush is white and DWM applies it before any softbuffer present reaches the screen. Eliminating it entirely requires either hooking WM_ERASEBKGND at a lower level than winit 0.29 exposes publicly, or using a GPU pipeline (Direct3D/Vulkan) which can present before the window is shown. Multiple approaches were attempted and each caused other problems. The flash is minimised but not eliminated.

No pan persistence across images

Pan resets to centre on every image navigation. There is no way to maintain a pan offset across images.

No status bar

Filename, index, and dimensions are not displayed. Adding them requires a font rasteriser crate (e.g. fontdue) and layout code — intentionally omitted for leanness.

Animated GIF only

The image crate's animation support covers GIF only. APNG and animated WebP/AVIF would need separate decoder paths.

Render loop is O(win_w × win_h)

The renderer iterates every window pixel on every redraw. Fast enough on any modern CPU but could be parallelised with rayon for very large windows.

monitor_usable falls back to heuristics on non-Windows

The SPI_GETWORKAREA path is Windows-only. Other platforms use 90%×85% of the monitor size.

JXL HDR clamped, not tone-mapped

f32_to_u8 clamps values above 1.0. A Reinhard or ACES tone mapping operator would produce better results for HDR JXL content.

SVG external resources not loaded

usvg with default options does not resolve external fonts or linked images. SVGs referencing these may render incompletely.

9. Glossary of Rust Concepts Used

Ownership
Every value has exactly one owner. When the owner goes out of scope the value is dropped. No garbage collector needed.

Borrowing
Lend a reference (&T;) without transferring ownership. Rust guarantees at compile time that references never outlive the value they point to.

&mut; self vs &self;
&mut; self = exclusive mutable access. &self; = shared read-only access. The compiler enforces that both cannot coexist simultaneously.

Option
Either Some(value) or None. Safe replacement for null pointers. You cannot use the value without explicitly handling the None case.

Result
Either Ok(value) or Err(error). The ? operator propagates errors up the call stack automatically.

Vec
A growable heap-allocated array.

HashMap
A hash table. O(1) average lookup, insert, and delete.

Arc
Atomically Reference Counted smart pointer. Shared ownership across threads. Value dropped when the last Arc is dropped.

Enum with data
Rust enums carry data per variant. Msg::Loaded carries path, image, bool. Msg::Error carries a string. Unlike C enums which carry no data.

From / Into
Standard conversion traits. impl From for Y lets you write x.into() and Rust calls the implementation.

then_some
(condition).then_some(value) returns Some(value) if true, None otherwise. Replaces if/else returning Option.

Lifetime
A compile-time label on a reference indicating how long it is valid. View<'a> borrows from App — the compiler ensures View cannot outlive App.

Closure
An anonymous function. move |event, elwt| { ... } moves ownership of captured variables — required for event_loop.run().

match
Exhaustive pattern matching. Every variant must be handled or the code won't compile.

#[derive(...)]
Auto-generates trait implementations. #[derive(Clone, Debug)] writes clone() and the debug formatter automatically.

Send
Marker trait: safe to transfer to another thread. Vec, u32, PathBuf, String, bool are all Send automatically.

ControlFlow
Winit enum. Wait sleeps until an event. WaitUntil sleeps until a deadline. Poll spins continuously.

EventLoopProxy
Cloneable, thread-safe handle to the event loop. proxy.send_event() injects a synthetic event from any thread.

NonZeroU32
A u32 guaranteed non-zero at compile time. softbuffer requires this for resize to prevent zero-size allocations.

cfg(windows)
Conditional compilation. Code in #[cfg(windows)] blocks compiles on Windows only.

unsafe
Allows calling C API functions the compiler cannot verify. Used for MessageBoxW and SystemParametersInfoW.

type alias
type Proxy = ...; creates a shorter name with no runtime cost.

Zero-size struct
struct WakeUp; has no fields and zero runtime size. Used as a type-level marker to parameterise EventLoop.